📂 Capa 1: Comunicación entre Componentes (Props, Emits y v-model)

En el desarrollo moderno con Vue 3 y Nuxt 4, la arquitectura se basa en **componentes**. Pero, ¿cómo hablan estos componentes entre sí sin crear un caos? Aquí es donde entran las tres herramientas fundamentales.

---

1. Props (Hacia Abajo ⬇️)

Las **Props** son el mecanismo para pasar datos de un componente "Padre" a un componente "Hijo".

🧠 El Concepto: "One-Way Data Flow"

Las props son de **solo lectura** para el hijo. Imagina que el padre le presta un libro al hijo: el hijo puede leerlo (usarlo), pero no puede rayarlo ni cambiarle el final (modificarlo). Si el libro debe cambiar, debe hacerlo el padre.

💻 Ejemplo Real en el Proyecto:

En `Track/List.vue` (Padre) pasamos la canción a `Track/Card.vue` (Hijo):

```
<!-- Track/List.vue (Padre) -->
<TrackCard :track="item" />

<!-- Track/Card.vue (Hijo) -->
<script setup>
const props = defineProps({
  track: {
    type: Object,
    required: true
  }
})
</script>
```

---

2. Emits (Hacia Arriba ⬆️)

Si el hijo no puede cambiar las props, ¿cómo le avisa al padre que el usuario hizo algo (como clic en borrar)? Usamos **Emits**.

🧠 El Concepto: "Aviso de Sucesos"

El hijo "grita" un evento y el padre, que está escuchando, decide qué hacer. Es como si el hijo levantara la mano en clase para decir "¡Terminé!". El profesor (padre) decide si le da el siguiente ejercicio.

💻 Ejemplo Real:

```
<!-- Track/Card.vue (Hijo) -->
<v-btn @click="$emit('delete', track.id)">Eliminar</v-btn>

<script setup>
```

```
const emit = defineEmits(['delete'])
</script>

<!-- Track/List.vue (Padre) -->
<TrackCard @delete="handleDelete" />
```

---

3. v-model y defineModel (Doble Vía ↔️)

A veces necesitamos que el dato fluya en ambas direcciones de forma constante, como en un campo de texto o un interruptor. Antes era complejo, pero Vue 3.4 introdujo `defineModel`.

🧠 El Concepto: "Sincronización Total"

`defineModel` crea una variable que está "atada" mágicamente entre el padre y el hijo. Si el hijo cambia el valor, el padre lo ve al instante, y viceversa. Es ideal para componentes de formulario o diálogos.

💻 Ejemplo Real en `Common/ConfirmDialog.vue`:

Este componente necesita que el padre controle si está abierto o cerrado, pero el hijo necesita cerrarse a sí mismo cuando el usuario hace clic en "Cancelar".

```
<!-- Common/ConfirmDialog.vue (Hijo) -->
<script setup>
const isVisible = defineModel() // Esto sincroniza el v-model del padre

function close() {
  isVisible.value = false // Esto cierra el diálogo tanto aquí como en el padre
}
</script>

<!-- Track/List.vue (Padre) -->
<CommonConfirmDialog v-model="confirmOpen" />
```

🚀 Resumen para Estudiantes:

Herramienta	Dirección	Analogía	Cuándo usar
Props	Padre -> Hijo	Prestar un objeto	Mostrar información.
Emits	Hijo -> Padre	Notificar una acción	Botones, eventos, clics.
defineModel	Bidireccional	Espejo sincronizado	Inputs, checkbox, diálogos.

⚠️ Error Común:

"Modificar una prop directamente". Si intentas hacer `props.track.title = 'Nuevo'`, Vue te dará un error en la consola. **No lo hagas**. Emite un evento o usa un Store si necesitas cambiar datos globales.